MANUAL TÉCNICO: SISTEMA DE GESTIÓN DE PARQUEO

Universidad de San Carlos de Guatemala (USAC) Juan Esteban Ruiz Villatoro 202400087

30 de octubre de 2026

1. Arquitectura Propuesta

El sistema está diseñado bajo el patrón **Modelo-Vista-Controlador (MVC)** modificado y adaptado para la gestión de estructuras de datos en memoria y concurrencia por hilos:

- **Capa de Modelo (`modelo`):** Contiene las entidades principales del negocio que almacenan la información básica e interactúan con las estructuras (clases `Residente` y `Vehiculo`).
- **Capa de Estructuras (`estructuras`):** Núcleo lógico del proyecto. Implementa de forma manual y desde cero todas las estructuras de datos dinámicas solicitadas (listas, colas, pilas), evitando el uso de colecciones prohibidas de Java (`ArrayList`, `LinkedList`, etc.).
- **Capa de Hilos (`hilos`):** Gestiona la concurrencia y la simulación en tiempo real de las garitas de entrada y salida mediante hilos (`Thread`), asegurando la sincronización de accesos compartidos.
- **Capa de Vista (`vista`):** Interfaz gráfica de usuario construida íntegramente con **Java Swing** y **AWT**, permitiendo la visualización en tiempo real del estado de los 150 espacios de estacionamiento, la gestión de residentes, el panel de control manual y los logs de eventos.

2. Diagrama de Clases (Estructura Lógica)

El sistema se compone de las siguientes relaciones principales entre clases y paquetes:

```
[ vista ] ---> utiliza ---> [ estructuras ] ---> gestionan ---> [ modelo ]

   VentanaPrincipal              ListaDobleResidentes        Residente
   VentanaGestionResidentes      ListaListasParqueo          Vehiculo
   VentanaParqueo                ColaVehiculos
                                   PilaVehiculos
[ hilos ] ---> manipulan ---> [ estructuras / modelo ]
   HiloGarita (Entrada)
   HiloGaritaSalida (Salida)
```

3. Estructuras de Datos y su Relación

Todas las estructuras han sido programadas manualmente mediante nodos enlazados:

1. **Lista Doblemente Enlazada de Residentes (`ListaDobleResidentes`):**
   - *Propósito:* Almacena de forma dinámica los residentes registrados en el sistema.

- *Relación:* Cada nodo (`NodoResidente`) contiene un objeto `Residente` y punteros `siguiente` y `anterior`. A su vez, cada residente posee una lista simple de vehículos asociados (`ListaSimpleVehiculos`).

2. **Lista Simple de Vehículos por Residente (`ListaSimpleVehiculos`):**
   - *Propósito:* Permite que un residente pueda registrar uno o varios vehículos vinculados a su perfil.
3. **Lista Doblemente Enlazada de Listas Simples (Parqueo - `ListaListasParqueo`):**
   - *Propósito:* Modela la distribución física del parqueo (150 espacios divididos en filas de la A a la I).
   - *Relación:* La estructura principal es una lista doble de filas (`FilaParqueo`), y cada fila contiene una lista simple de espacios físicos individuales (`EspacioParqueo`).
4. **Cola de Vehículos (`ColaVehiculos / ColaSalida`):**
   - *Propósito:* Administra el flujo FIFO (First In, First Out) de los automóviles que ingresan o egresan por las garitas de control.
5. **Pila de Eventos (`PilaVehiculos`):**
   - *Propósito:* Registra el historial de transacciones, ingresos y salidas bajo una estructura LIFO (Last In, First Out) para la visualización en el panel de eventos.

4. Flujo de Entrada y Salida de Vehículos

4.1. Flujo de Entrada:

1. **Llegada / Encolamiento:** El vehículo llega al sistema y es colocado en la `ColaVehiculos` de entrada.
2. **Procesamiento por Hilos (Garitas 1, 2 y 3):** Los hilos activos (`HiloGarita`) simulan el tiempo de atención de cada garita de forma concurrente.
3. **Validación de Reglas de Negocio:**
   - Se extrae la placa del vehículo.
   - Se consulta la `ListaDobleResidentes` mediante el método `esVehiculoDeSocio(placa)`.
   - Si el destino es el **Área A–C** (exclusiva para socios), el sistema verifica de forma estricta que el residente tenga el estatus `isEsSocio() == true`. De lo contrario, se rechaza el acceso y se registra en la pila de eventos.
4. **Asignación de Espacio:** Si cumple las validaciones, el vehículo es enlazado al primer `EspacioParqueo` libre disponible en la sección correspondiente.

4.2. Flujo de Salida:

1. **Solicitud de Retiro:** Se ingresa la placa del vehículo a retirar (de forma manual o mediante las garitas de salida).
2. **Búsqueda y Liberación:** La estructura `ListaListasParqueo` busca el espacio ocupado por dicha placa (`liberarEspacioVehiculo(placa)`).
3. **Actualización de Estado:** El espacio pasa de estado ocupado (`isOcupado = true`) a libre (`isOcupado = false`), y se registra el evento en la `PilaVehiculos`.

5. Uso de Hilos y Sincronización

- **Concurrencia:** Se implementan múltiples hilos independientes (`HiloGarita` para las 3 entradas y `HiloGaritaSalida` para las 2 salidas) que heredan de la clase `Thread` de Java.
- **Sincronización:** Para evitar condiciones de carrera (*race conditions*) al momento de modificar el estado de los espacios de parqueo compartidos o las colas de espera, los métodos críticos de asignación y liberación cuentan con bloques sincronizados (`synchronized`) o mecanismos de control de concurrencia sobre los punteros de las estructuras.

6. Organización de Paquetes

El código fuente se encuentra distribuido de manera modular en los siguientes paquetes dentro del directorio `src`:

- `estructuras`: Contiene todas las clases de nodos y estructuras de datos manuales (`ListaDobleResidentes`, `ListaListasParqueo`, `ColaVehiculos`, `PilaVehiculos`, etc.).
- `modelo`: Clases que representan los objetos de dominio (`Residente`, `Vehiculo`).
- `vista`: Interfaces gráficas desarrolladas en Swing (`VentanaPrincipal`, `VentanaGestionResidentes`, etc.).
- `hilos`: Clases de control concurrente para las garitas (`HiloGarita`, `HiloGaritaSalida`).

7. Formato de los Archivos de Persistencia

El sistema opera mediante archivos de texto plano estructurados para la lectura y escritura secuencial de los datos del sistema:

- `residentes.txt`**:** Almacena la información de los residentes registrados:

```
ID;Nombre;NumeroCasa;EsSocio
```

- `vehiculos.txt`**:** Almacena el registro histórico o asociado de los vehículos vinculados a los residentes o al sistema de parqueo.